

Bobine QA Verification — Features 01 through 03

Walk through each section in order. Check every item. If something fails, note the failure and continue — don't stop the QA pass. Compile all failures at the end.

Prerequisites

Before starting:

- ☐ Fresh `mix ecto.reset` — clean database with seed data
 - ☐ `mix test --exclude e2e` — all tests passing
 - ☐ Dev server running: `mix phx.server`
 - ☐ Mux webhook tunnel running: `ngrok http 4000` with URL configured in Mux dashboard
-

Feature 01: Multi-Tenant Resolution and Admin Dashboard Shell

1.1 Seed Data

- ☐ Running `mix ecto.reset` seeds at least two organizations
- ☐ Seeds create a super admin user
- ☐ Seeds create admin, editor, and viewer_support users for the primary org
- ☐ Seeds create a default theme for each org

1.2 Organization Resolution

- ☐ Loading the app resolves to an organization (via dev fallback)
- ☐ The resolved org name appears somewhere in the admin layout
- ☐ If you change the dev fallback to a non-existent org, you get a 404

1.3 Authentication

- ☐ Unauthenticated user cannot access `/admin`
- ☐ Unauthenticated user is redirected to login
- ☐ User can log in via magic link flow
- ☐ After login, user is redirected to the admin dashboard

1.4 Admin Dashboard Shell

- ☐ `/admin` renders the admin dashboard

☐ Sidebar navigation is visible with links to all 8 admin sections:

☐ Dashboard

☐ Content

☐ Catalog

☐ Analytics

☐ Branding

☐ Members

☐ Webhooks

☐ Settings

☐ Clicking each sidebar link navigates to the correct page

☐ The current page is highlighted in the sidebar

☐ Each page renders its title (even if it's a placeholder)

1.5 RBAC Enforcement

☐ Owner can access `/admin`

☐ Admin can access `/admin`

☐ Editor can access `/admin`

☐ Viewer_support can access `/admin`

☐ User with no membership in the org cannot access `/admin`

☐ User from a different org cannot access this org's admin

1.6 Viewer Placeholder Pages

☐ `/` (homepage) renders with the org name

☐ `/watch/:id` exists (can be a placeholder)

☐ `/watchlist` exists (can be a placeholder)

☐ `/account` exists (can be a placeholder)

Feature 02: Super Admin Panel

2.1 Super Admin Access

☐ Super admin user can access `/super`

☐ Non-super-admin user is redirected away from `/super`

☐ Unauthenticated user is redirected away from `/super`

2.2 Super Admin Dashboard

☐ `/super` shows platform stats (total orgs, users, videos)

- ☐ Stats are accurate based on seed data

2.3 Organization Management

- ☐ `/super/organizations` lists all organizations
- ☐ Each org row shows name, slug, and member/video counts
- ☐ Search input filters orgs by name or slug
- ☐ "New Organization" button is visible and links to the form
- ☐ `/super/organizations/new` renders the creation form
- ☐ Creating an org with valid data succeeds and redirects to show page
- ☐ Creating an org with a duplicate slug shows a validation error
- ☐ Creating an org with missing name shows a validation error
- ☐ The new org gets a default theme created automatically
- ☐ The specified owner email gets an owner membership created
- ☐ `/super/organizations/:id` shows org details
- ☐ Edit button links to edit form
- ☐ Editing org name/slug succeeds
- ☐ `/super/organizations/:id` shows member list

2.4 User Management

- ☐ `/super/users` lists all users
- ☐ Super admin users have a visible badge
- ☐ Grant super admin action works (with confirmation)
- ☐ Revoke super admin action works (with confirmation)
- ☐ Cannot revoke your own super admin status

2.5 Impersonation

- ☐ "Open as Admin" button is visible on org show page
 - ☐ Clicking it navigates to `/admin` in the context of that org
 - ☐ An impersonation banner is visible during impersonation
 - ☐ The banner shows which org is being impersonated
 - ☐ "Stop impersonating" button returns to the super admin panel
 - ☐ While impersonating, the admin dashboard shows the correct org's data
 - ☐ Non-super-admin users cannot trigger impersonation
-

Feature 02.1: Architecture Retrofit

2.1.1 Audit Logging

- ☐ Creating a resource (e.g. a new org via super admin) writes an audit log
- ☐ Audit log contains: action, resource_type, resource_id, user_id, org_id
- ☐ Audit log contains metadata (request_id, ip) when available
- ☐ Impersonation actions include `impersonated_by` in metadata
- ☐ Audit logs have `inserted_at` but no `updated_at`

2.1.2 Request Context

- ☐ `Bobine.RequestContext.current()` returns context during a request
- ☐ Context includes request_id, ip, and user_agent
- ☐ Context is nil outside of a request (e.g. in iex)

2.1.3 Soft Deletes

- ☐ Deleting a video (when Feature 03 is built) sets `deleted_at` instead of removing the record
- ☐ Deleted records don't appear in list queries
- ☐ Verify in iex: `Repo.all(Video)` returns deleted records, but `Content.list_videos(org)` does not
- ☐ Deleted orgs still resolve for the purpose of showing a deactivation page (not a 404)

2.1.4 Pagination

- ☐ Context list functions return `%{results: [...], page: _, per_page: _, total: _, total_pages: _}`
- ☐ Passing `page: 2, per_page: 1` returns the second item
- ☐ `per_page` is clamped to max 100
- ☐ Empty results return `total: 0, total_pages: 1`

2.1.5 Event Broadcasting

- ☐ Creating a resource broadcasts an event on PubSub
- ☐ Verify in iex:

elixir

```
Bobine.Events.subscribe("org-id-here")
# Then trigger a create action in another process/tab
# Should receive {:bobine_event, {event, resource}, scope}
```

- ☐ The AuditSubscriber is running in the supervision tree
- ☐ Events result in audit log entries being written

2.1.6 Consistent Error Tuples

- ☐ Context functions return `{:error, :validation, changeset}` (not bare `{:error, changeset}`)
- ☐ Context functions return `{:error, :not_found}` for missing records
- ☐ Verify in iex by calling a context function with bad input

2.1.7 Feature Flags

- ☐ `Bobine.Features.enabled?/2` returns false for unset features
- ☐ Setting a feature in the org's features map and calling `enabled?/2` returns true
- ☐ Verify in iex:

elixir

```
org = Repo.get_by(Organization, slug: "demo")
Bobine.Features.enabled?(org, :live_streaming) # => false
```

2.1.8 Idempotency

- ☐ `Bobine.Idempotency.key/3` returns a deterministic string
- ☐ Same inputs on the same day produce the same key

2.1.9 Data Export

- ☐ `Bobine.Admin.export_organization_data/1` returns a map with all tenant-scoped data
 - ☐ Export includes data from the target org only
 - ☐ Export includes soft-deleted records
-

Feature 02.2: OpenTelemetry Instrumentation

2.2.1 Auto-Instrumentation

- ☐ Loading a page in dev produces span output in the terminal (stdout exporter)
- ☐ Span output includes Phoenix request spans
- ☐ Span output includes Ecto query spans nested under the request
- ☐ Triggering a LiveView event produces spans

2.2.2 Custom Spans

- ☐ Context mutation functions (e.g. creating a video) produce custom spans named `bobine.<context>.<operation>`
- ☐ Spans include `bobine.org.id` attribute for org-scoped operations

2.2.3 Metrics

- ☐ `Bobine.Metrics` module exists with functions for business events
- ☐ Calling a metrics function emits a `:telemetry` event (verify in iex with `:telemetry_test`)

2.2.4 Structured Logging

- ☐ Logger output includes `request_id` in metadata
- ☐ Logger output includes `org_id` when an org is in scope
- ☐ Oban workers set `org_id` in Logger metadata

2.2.5 Grafana Cloud Connection (if deployed)

- ☐ Traces appear in Grafana Cloud Tempo after performing actions on the deployed app
 - ☐ Traces show service name "bobine"
 - ☐ Clicking into a trace shows the full span waterfall
-

Feature 03: Mux Video Upload and Playback

3.1 Mux Client

- ☐ `Bobine.Content.MuxClientBehaviour` exists with callbacks defined
- ☐ `Bobine.Content.MuxClient` implements the behaviour
- ☐ Mock is registered in `test/support/mocks.ex`
- ☐ Test config uses the mock: `config :bobine, :mux_client, MockMuxClient`

3.2 Video Upload Flow

- ☐ Navigate to `/admin/content`
- ☐ "Upload Video" button is visible
- ☐ Clicking it opens an upload modal/form
- ☐ Enter a title and select a video file
- ☐ Submit the form
- ☐ A video record appears in the list with status "waiting"
- ☐ The browser uploads the file directly to Mux (check Network tab — PUT to `storage.googleapis.com`)
- ☐ Video bytes never hit the Bobine server (no large POST to localhost)
- ☐ Upload progress is visible (percentage or progress bar)
- ☐ After upload completes, status changes to "preparing"

3.3 Mux Webhook Processing

- ☐ Mux webhook arrives at `/webhooks/mux` (visible in ngrok and Phoenix logs)
- ☐ Webhook is accepted with 200 response
- ☐ An Oban job is enqueued for processing (check in iex: `Oban.drain_queue(:mux)` or check logs)
- ☐ `video.upload.asset_ready` webhook links the upload to the asset
- ☐ `video.asset_ready` webhook updates video status to "ready"
- ☐ Video record now has: `mux_asset_id`, `mux_playback_id`, `duration`
- ☐ The content list page updates the status badge in real-time (no manual refresh needed)

3.4 Video Status State Machine

- ☐ New video starts with status "waiting"
- ☐ After upload + asset creation, status becomes "preparing"
- ☐ After encoding complete, status becomes "ready"
- ☐ If Mux reports an error, status becomes "errored"
- ☐ Verify in iex that the video record has the correct status at each stage

3.5 Content Management Page

- ☐ `/admin/content` lists all videos for the current org
- ☐ Each video shows: thumbnail, title, status, duration, upload date
- ☐ Thumbnails load from Mux: `https://image.mux.com/{playback_id}/thumbnail.webp`
- ☐ Ready videos show formatted duration (mm:ss)
- ☐ Waiting/preparing videos show status badge instead of duration
- ☐ Search input filters videos by title
- ☐ Videos from other orgs are NOT visible
- ☐ Empty state is shown when no videos exist
- ☐ Delete button exists on each video
- ☐ Deleting a video soft-deletes it (disappears from list, still in DB)
- ☐ Deleting triggers a background job to clean up the Mux asset

3.6 Video Playback Page

- ☐ Navigate to `/watch/:id` for a ready video
- ☐ Page renders the video title and description
- ☐ Mux Player web component is present in the DOM
- ☐ Player has the correct `data-playback-id` attribute
- ☐ Video plays when you click play
- ☐ Navigating to a non-existent video redirects to homepage

- ☐ Navigating to a video with status != "ready" redirects to homepage

3.7 Playback Progress

- ☐ Play a video for at least 30 seconds
- ☐ Check that progress is being reported (look for `playback_progress` events in logs or LiveView events)
- ☐ Progress writes go to a buffer, NOT directly to Postgres (check logs — should not see an INSERT for every progress event)
- ☐ Stop the video and navigate away
- ☐ Return to the same video — playback resumes from the saved position
- ☐ The `data-resume-position` attribute reflects the saved position

3.8 TypeScript Hooks

- ☐ No JavaScript errors in the browser console during upload
- ☐ No JavaScript errors in the browser console during playback
- ☐ `npx tsc --noEmit --project assets/tsconfig.json` passes with no errors

3.9 Multi-Tenant Isolation

- ☐ Upload a video in org A
- ☐ Switch to org B (via super admin impersonation or dev fallback)
- ☐ Org B's content page does NOT show org A's video
- ☐ Org B cannot access org A's video via direct URL (`/watch/:org_a_video_id`)

3.10 RBAC on Content Management

- ☐ Admin role can upload videos
 - ☐ Editor role can upload videos
 - ☐ Viewer_support role CANNOT upload videos (no upload button, or action denied)
 - ☐ Viewer_support CAN view the content list (read-only)
-

Cross-Cutting Concerns

Data-Test Attributes

- ☐ All interactive elements have `data-test` attributes
- ☐ Spot-check: upload button, video rows, delete buttons, search input, sidebar nav links

Error States

- ☐ Submitting the upload form with no title shows a validation error
- ☐ Submitting the upload form with no file shows an error (client-side)
- ☐ Invalid webhook signature returns 400 (test with curl and a bad payload)

Code Quality

- ☐ `mix format --check-formatted` passes
 - ☐ `mix credo --strict` passes
 - ☐ `mix dialyzer` passes (or note known warnings)
 - ☐ `npx tsc --noEmit` passes
 - ☐ `mix test --exclude e2e` — all tests pass
 - ☐ `mix test --only e2e` — all E2E tests pass (requires Chrome + ChromeDriver)
-

Failure Log

Record any failures here:

#	Feature	Step	Expected	Actual	Notes
1					
2					
3					
4					
5					

Sign-off

- ☐ All features verified
- ☐ All failures logged
- ☐ Tickets created for any failures
- ☐ Ready to proceed to Feature 04 (Content Management)